Projects Proposal

This document contains a list of the possible projects (worth 40% of the total grading) for the topic Seguretat i Secret en la Codificació de la Informació.

(i) Organize yourself in teams of 2 to 3 people (preferably pairs, but teams of 3 are allowed).

(ii) Choose a project to do and let the teacher know of your choice; if the project is available, the teacher will assign it to you.

(iii) Work on the project during November, December, Christmas break, and January.

(iv) Choose a presentation slot from the available days in January.

(v) Deliver a written version of the project and play an oral presentation (of no more than one hour, including 10-15 minutes for questions).

(vi) Each of the written document, the text of the (power-point or similar) presentation, and the oral exposition can be in English and/or in catalan.

(vii) The members of the team can internally organize themselves as they wish (all tasks done by all members, separate parts of the project to each member, etc) but all the members are responsable for all the project. That is, in the oral presentation everybody must be able to defend and answer questions about any aspect of the project.

(viii) The grading will take into account both the written document (completeness, correctness, good presentation, etc), and the quality of the oral presentation (organization, ability to express, maturity in the topic, how do you defend yourself answering questions, etc); if necessary, this second part may be evaluated with different marks for the different members of the team.

There are two type of projects: topic projects and computer projects:

A. The topic projects consist on looking for information about the proposed topic (books, bibliography, internet, etc), understanding and making sense of it, and then writing a document and preparing a 1-hour oral presentation, covering the aspects required, and assuming as background all what you learned along the course but not more. That is, as a reference for the tone and level of your text/presentation, imagine you are the teacher of this course, and the audience are your classmates in this same course.

B. The program projects consist on writing a computer programe for doing the required tasks (typically about one of the cryptosystems or codes studied along the course). You can freely choose the programming language you prefer but, in case they exist, you cannot call libraries or packages doing essentially what you are supposed to do. The program must work correctly in all cases, must give no errors, and you must show it in action during your oral presentation. The written document must contain a first part explaining the foundations and the basics of the method you are working with, and a second part including all the code you wrote and explanations on how is it organized and how is each part/routine working.

Índex

1 PROGRAM: Linear Codes in action (text) 3

2 PROGRAM: Hamming Codes in action (images) 4

3 TOPIC: Non-block Codes 5

4 TOPIC: Study of binary Golay codes 6

5 TOPIC: Study of the Reed–Muller codes 7

6 PROGRAM: The k-dimensional affine cryptosystem 8

7 PROGRAM: Prime numbers and primality tests 9

8 PROGRAM: The RSA cryptosystem 10

9 PROGRAM: The Rabin cryptosystem 11

10 PROGRAM: The El Gamal cryptosystem 12

1 PROGRAM: Linear Codes in action (text)

Goal: Implementation of the use of Linear Codes.

(i) Write a survey explaining what is a linear code, which interesting matrices does it have associated and what are them useful for, what are their parameters, how to codify, how to decodify, and how to detect/correct errors.

(ii) Implement a routine for computing the canonical generating and control matrices for a linear code:

$$\begin{cases} \text{Input:} & n \in \mathbb{N}, \text{ and a set of generators} \\ & \mathbf{v_1}, \ldots, \mathbf{v_k} \in (\mathbb{Z}/2\mathbb{Z})^n \text{ for } C; \\ \text{Output:} & \text{the canonical generating and control matrices,} \\ & G \text{ and } H, \text{ for } C. \end{cases}$$

(iii) Implement a routine for computing the parameters of a linear code C:

$$\begin{cases} \text{Input:} & n \in \mathbb{N}, \text{ the canonical generating matrix } G \\ & \text{for } C; \\ \text{Output:} & \text{the length, dimension, size, delta, and} \\ & \text{detection and correction capabilities for } C. \end{cases}$$

(iv) Implement a routine for codifying with a linear code C:

$$\begin{cases} \text{Input:} & n \in \mathbb{N}, \text{the canonical generating matrix } G \\ & \text{for } C, \text{ and a text in catalan;} \\ \text{Output:} & \text{the codified list of bits.} \end{cases}$$

(v) Implement a routine for correcting the possible errors plus decodifying with a linear code C:

$$\begin{cases} \text{Input:} & n \in \mathbb{N}, \text{the canonical generating matrix } G \\ & \text{for } C, \text{ and the codified list of bits;} \\ \text{Output:} & \text{correction of errors, and decodified text in catalan.} \end{cases}$$

2 PROGRAM: Hamming Codes in action (images)

Goal: Implementation of the use of Hamming Codes in image files.

(i) Write a survey explaining what is a Hamming code, which interesting matrices does it have associated and what are them useful for, what are their parameters, how to codify, how to decodify, and how to detect/correct errors.

(ii) Implement a routine for computing the canonical generating and control matrices for a Hamming code:

$$\begin{cases} \text{Input:} & t \in \mathbb{N}; \\ \text{Output:} & \text{the canonical generating and control matrices for } Ham(t). \end{cases}$$

(iii) Implement a routine for codifying with $Ham(t)$:

$$\begin{cases} \text{Input:} & t \in \mathbb{N}, \text{and a list of bits;} \\ \text{Output:} & \text{the codified list of bits.} \end{cases}$$

(iv) Implement a routine for correcting the possible errors plus decodifying with $Ham(t)$:

$$\begin{cases} \text{Input:} & t \in \mathbb{N}, \text{and the codified list of bits;} \\ \text{Output:} & \text{correction of errors, and decodified list of bits.} \end{cases}$$

(v) Implement a routine for codifying an image file with $Ham(t)$:

$$\begin{cases} \text{Input:} & t \in \mathbb{N}, \text{and an image file;} \\ \text{Output:} & \text{the codified file.} \end{cases}$$

(vi) Implement a routine for introducing random errors into an image file:

$$\begin{cases} \text{Input:} & t \in \mathbb{N}, \alpha \in (0,1), \text{and an image file;} \\ \text{Output:} & \text{the image file corrupted at rate } \alpha. \end{cases}$$

(vii) Implement a routine for correcting the possible errors plus decodifying with $Ham(t)$:

$$\begin{cases} \text{Input:} & t \in \mathbb{N}, \text{and a corrupted image;} \\ \text{Output:} & \text{correction of errors, and the recovered image.} \end{cases}$$

3 TOPIC: Non-block Codes

Goal: Study the possibility of doing Coding Theory with non-block çodes" (i.e. containing words of different lengths).

(i) Define the notion of free monoid over an alphabet A, and the general notion of (non-block) code. Explain the case of the prefix sets and show examples.

(ii) Explain Sardinas–Patterson's Theorem and look for information about its proof. Explain how to find an ambiguous message in case it exists.

(iii) Explain Kraft inequality (including a proof) and discuss the validity of its converse.

(iv) Give equivalent conditions for a prefix code to be maximal (including proofs).

(v) Solve at least half of the exercises from Chapter 3 in the book below.

REFERENCES: Mainly Chapter 3 in the textbook *Informació i codis*, J.M. Brunat, E. Ventura, Edicions UPC, Barcelona, 1st edition, 2001.

4 TOPIC: Study of binary Golay codes

Goal: To describe the binary Golay codes G_{24} and G_{23}, and to understanding their properties and error detection and correction algorithms.

(i) Definition of autodual code and anti-orthogonal matrix. Show the relation among these concepts.

(ii) Define the Golay binary codes G_{24} and G_{23}.

(iii) Find a generator and control matrices G and H for G_{24}. Compute the parameters of this code; which are its detection and correction capabilities?

(iv) Explain how can we use the syndromes with respect to matrices G and H to detect and correct errors with G_{24}. Do the full details in an example.

(v) Compute the parameters of the Golay code G_{23} and prove it is a perfect code.

(vi) Explain an algorithm for detecting and correcting errors with G_{23}.

(vii) **(Optional)** Implement an algorithm to codify and correct+decodify with G_{24} and G_{23}.

REFERENCES: Mainly Section 11.3 in the textbook *Informació i codis*, J.M. Brunat, E. Ventura, Edicions UPC, Barcelona, 1st edition, 2001.

5 TOPIC: Study of the Reed–Muller codes

Goal: To describe the binary Reed–Muller codes $\mathcal{RM}(r, m)$, and to understanding their properties and error detection and correction algorithms.

(i) Define reduction of multivariate polynomials and the evaluation function; study the first properties of these notions.

(ii) Define the Reed–Muller codes and study their first properties.

(iii) Compute the parameters of Reed–Muller codes; which are their detection and correction capabilities?

(iv) Explain an algorithm for detecting and correcting errors with a Reed–Muller code. Do the full details in an example.

(v) **(Optional)** Implement an algorithm to codify and correct+decodify with a Reed–Muller code.

REFERENCES: Mainly Chapter 12 in the textbook *Informació i codis*, J.M. Brunat, E. Ventura, Edicions UPC, Barcelona, 1st edition, 2001.

6 PROGRAM: The k-dimensional affine cryptosystem

Goal: Implementation of the k-dimensional affine cryptosystem.

(i) Write a survey explaining how the k-dimensional affine cryptosystem works: what are the private keys, how to encrypt, how to decrypt, and why does it altogether work correctly.

(ii) Implement a routine with an algorithm for Inversion of Modular Matrices:

$$\begin{cases} \text{Input:} & n, k \in \mathbb{N},\ A \in \mathrm{Mat}_k(\mathbb{Z}/n\mathbb{Z}); \\ \text{Output:} & \text{the inverse matrix } A^{-1} \pmod{n} \text{ if it exists,} \\ & \text{and an alarm if not.} \end{cases}$$

(iii) Implement a routine with the k-Dimensional Affine Encrypting Module:

$$\begin{cases} \text{Input:} & \text{the secrete key } n \in \mathbb{N},\ A \in \mathrm{Mat}_k(\mathbb{Z}/n\mathbb{Z}),\ b \in (\mathbb{Z}/n\mathbb{Z})^k, \\ & \text{and a text written in catalan;} \\ \text{Output:} & \text{the encrypted message.} \end{cases}$$

(iv) Implement a routine with the k-Dimensional Affine Decrypting Module:

$$\begin{cases} \text{Input:} & \text{the secrete key } n \in \mathbb{N},\ A \in \mathrm{Mat}_k(\mathbb{Z}/n\mathbb{Z}),\ b \in (\mathbb{Z}/n\mathbb{Z})^k, \\ & \text{and the encrypted text;} \\ \text{Output:} & \text{the original text (or an alarm if } A \text{ is not invertible).} \end{cases}$$

(v) Implement a routine to break the k-Dimensional Affine Cryptosystem:

$$\begin{cases} \text{Input:} & k, \text{ and } k+1 \text{ codified vectors} \\ & (X_0, Y_0), (X_1, Y_1), \ldots, (X_k, Y_k) \in (\mathbb{Z}/n\mathbb{Z})^{2k}; \\ \text{Output:} & \text{the secrete key } A,\ b, \\ & \text{or an alarm if the data is not valid.} \end{cases}$$

7 PROGRAM: Prime numbers and primality tests

Goal: To study the problem of generating big prime numbers, and implementation of some primality tests.

(i) Define prime number and show there are infinitely many (even adding some extra congruence conditions).

(ii) Explain the root criterium and the sieve method to find prime numbers.

(iii) Implement two algorithms to find all primes up to a given bound; one by doing brute force divisions, and the other following the Sieve method:

$$\begin{cases} \text{Input:} & n \in \mathbb{N}; \\ \text{Output:} & \text{a list of all primes up to } n. \end{cases}$$

Compare the efficiency of both algorithms run in your computer.

(iv) Explain and proof Fermat's Theorem, define Carmichael numbers, give examples, and explain Fermat's test for primality.

(v) Implement Fermat's (probabilistic) test for primality:

$$\begin{cases} \text{Input:} & n \in \mathbb{N}; \\ \text{Output:} & \text{yes/no whether } n \text{ is prime.} \end{cases}$$

(v) Explain Miller–Rabin's test for primality, give examples, and compare with Fermat's test.

(vi) Implement Miller–Rabin's (probabilistic) test for primality:

$$\begin{cases} \text{Input:} & n \in \mathbb{N}; \\ \text{Output:} & \text{yes/no whether } n \text{ is prime.} \end{cases}$$

d) Implement a (probabilistic) Random Prime Generator:

$$\begin{cases} \text{Input:} & k \in \mathbb{N}; \\ \text{Output:} & \text{a prime number of } k \text{ digits.} \end{cases}$$

REFERENCES: Mainly Chapter 6 in the textbook *Introduction to cryptography*, J.A. Buchmann, Springer (2000).

8 PROGRAM: The RSA cryptosystem

Goal: Implementation of the RSA cryptosystem.

(i) Write a survey explaining how the RSA cryptosystem works: what are the public and private keys, how to encrypt, how to decrypt, and why does it altogether work correctly.

(ii) Implement a routine with the Extended Euclidean Algorithm to compute inverses in modular arithmetic:

$$\begin{cases} \text{Input:} & n, a \in \mathbb{N}; \\ \text{Output:} & \alpha, \beta \in \mathbb{Z} \text{ such that} \\ & \alpha n + \beta a = \text{mcd}(n, a). \end{cases}$$

(iii) Implement a routine with the Fast Modular Exponentiation algorithm:

$$\begin{cases} \text{Input:} & n \in \mathbb{N},\ M \in \mathbb{Z}/n\mathbb{Z},\ e \in \mathbb{N}; \\ \text{Output:} & \text{the value of } M^e \pmod{n}. \end{cases}$$

(iv) Implement a routine with the RSA Generation Key Algorithm:

$$\begin{cases} \text{Input:} & p, q \in \mathbb{N},\ \text{primes}; \\ \text{Output:} & n = pq \text{ and valid values for } e, d. \end{cases}$$

(v) Implement the Encrypting Module:

$$\begin{cases} \text{Input:} & \text{the public key } n, e \in \mathbb{Z}, \text{ and} \\ & \text{a text written in catalan}; \\ \text{Output:} & \text{the encrypted message}. \end{cases}$$

(vi) Implement the Decrypting Module:

$$\begin{cases} \text{Input:} & n, d, \text{ and the encrypted message}; \\ \text{Output:} & \text{the decrypted message}. \end{cases}$$

9 PROGRAM: The Rabin cryptosystem

Goal: Implementation of the Rabin cryptosystem.

(i) Write a survey explaining how the Rabin cryptosystem works: what are the public and private keys, how to encrypt, how to decrypt, and why does it altogether work correctly; explain the small possible ambiguity and how to overcome it.

(ii) Implement a routine with the Extended Euclidean Algorithm:

$$\begin{cases} \text{Input:} & p, q \in \mathbb{N}; \\ \text{Output:} & \alpha, \beta \in \mathbb{Z} \text{ such that} \\ & \alpha p + \beta q = \text{mcd}(p, q). \end{cases}$$

(iii) Implement a routine checking whether a given prime number is Blum or not

$$\begin{cases} \text{Input:} & p \in \mathbb{N}; \\ \text{Output:} & \text{yes/no about } p \text{ being Blum.} \end{cases}$$

(iv) Implement a routine with the Fast Modular Exponentiation algorithm:

$$\begin{cases} \text{Input:} & n \in \mathbb{N},\ C \in \mathbb{Z}/n\mathbb{Z},\ e \in \mathbb{N}; \\ \text{Output:} & \text{the value of } C^e \pmod{n}. \end{cases}$$

(v) Implement the Encrypting Module:

$$\begin{cases} \text{Input:} & \text{the public key } n \in \mathbb{Z}, \text{ and} \\ & \text{a text written in catalan;} \\ \text{Output:} & \text{the encrypted message.} \end{cases}$$

(vi) Implement the Decrypting Module:

$$\begin{cases} \text{Input:} & p, q \text{ and the encrypted message;} \\ \text{Output:} & \text{an alarm if } p \text{ or } q \text{ are not Blum,} \\ & \text{and the decrypted message otherwise.} \end{cases}$$

10 PROGRAM: The El Gamal cryptosystem

Goal: Implementation of the El Gamal cryptosystem.

(i) Write a survey explaining how the El Gamal cryptosystem works: what are the public and private keys, how to encrypt, how to decrypt, and why does it altogether work correctly.

(ii) Implement a routine with the Fast Modular Exponentiation algorithm:

$$\begin{cases} \text{Input:} & p \in \mathbb{N},\ g \in \mathbb{Z}/p\mathbb{Z},\ e \in \mathbb{N}; \\ \text{Output:} & \text{the value of } g^e \pmod{p}. \end{cases}$$

(iii) Implement a routine checking Root Primitivity:

$$\begin{cases} \text{Input:} & p \in \mathbb{N} \text{ prime},\ g \in \mathbb{Z}/p\mathbb{Z}; \\ \text{Output:} & \text{yes/no whether } g \text{ is a primitive root mod } p. \end{cases}$$

(iv) Implement a routine with the El Gamal Encrypting Module:

$$\begin{cases} \text{Input:} & \text{the public key } p \in \mathbb{Z}, g, g^x \in \mathbb{Z}/p\mathbb{Z}, \\ & \text{and a text written in catalan;} \\ \text{Output:} & \text{the encrypted message.} \end{cases}$$

(v) Implement the Decrypting Module:

$$\begin{cases} \text{Input:} & p, x, \text{ and the encrypted message;} \\ \text{Output:} & \text{the decrypted message.} \end{cases}$$